INTELLIGENT TRAFFIC MANAGEMENT LUXEMBOURG

Partie XAI : expliquer les décisions de l'agent Q-learning S3

Ce que nous avons réalisé, comment l'interface l'affiche, et exactement quoi cliquer et quoi dire pendant la démonstration.

Zone étudiée	Kirchberg (Luxembourg-Ville), 4 carrefours à feux de l'avenue J.F. Kennedy
Données	Simulation SUMO, scénario LuST (vers 2015), 07:00–08:00 simulé — aucune mesure réelle
Agent expliqué	S3 : agent d'apprentissage par renforcement, Q-learning tabulaire, table gelée
Méthode XAI	Valeurs de Shapley exactes (SHAP), calculées par notre module `agents/xai/shap_qtable.py`
Interface	Page « Explicabilité XAI » (Next.js) reliée au backend Python FastAPI par les routes `/api/xai/*`

Sommaire

1. L'essentiel en une page
2. Rappel : comment l'agent S3 décide
3. SHAP expliqué simplement
4. Ce que nous avons réalisé, étape par étape
5. Comment l'interface affiche l'XAI (architecture)
6. La page « Explicabilité XAI » pièce par pièce
7. Préparer la démonstration (la veille et le jour même)
8. Scénario de démonstration : je clique sur quoi, je dis quoi
9. Plan B : si quelque chose ne marche pas
10. Questions probables du professeur et réponses
11. Limites à dire honnêtement
12. Lexique des mots techniques

Document du 30 octobre 2026. Les captures d'écran ont été faites sur une copie de l'application avec un vrai run SUMO de la stratégie S3 ; sur votre PC, l'heure, le nombre de décisions et les valeurs exactes du run lancé en direct peuvent être différents. Les valeurs de l'exemple de référence (07:29:01, Carrefour 4) sont, elles, toujours les mêmes.

1. L'essentiel en une page

Le problème. Notre agent S3 apprend seul, par renforcement, quand terminer le vert d'un feu. Il décide à partir d'une **table Q** : pour chaque situation, deux nombres (valeur de « garder » et valeur de « passer »). Ces nombres ne disent pas *pourquoi* l'agent a choisi. C'est une « boîte noire » : l'objectif de la partie XAI (*eXplainable AI*, IA explicable) est de l'ouvrir.

La solution. Pour chaque décision prise par la table, nous calculons les **valeurs de Shapley exactes** (méthode SHAP) : elles disent de combien chacune des 4 variables observées a poussé l'agent vers « passer » ou vers « garder », par rapport à une situation de référence.

Ce que nous avons construit

- **Un moteur SHAP exact** (`shap_qtable.py`) : calcul sans approximation, vérification des valeurs contre la table, additivité contrôlée.
- **Des analyses scientifiques** : stabilité selon la pondération du fond de référence, figures, revue scientifique, étude sur plusieurs décisions.
- **Une page web dynamique « Explicabilité XAI »** : elle montre les décisions réelles pendant la simulation, et calcule à la demande le SHAP de la décision choisie.
- **Des routes API** `/api/xai/runs` , `/api/xai/decisions` , `/api/xai/shap` dans le backend FastAPI.
- **Des tests automatisés** : 18 + 11 + 12 + 19 tests pour les modules XAI, 9 tests pour l'API, 26 tests de l'interface.

> **LA PHRASE À RETENIR**
>
> « Pour chaque décision prise par la table Q-learning, nous calculons exactement de combien chacune des quatre variables observées a poussé l'agent à passer au vert suivant ou à garder le vert, par rapport à une situation de référence. Le calcul est exact, vérifié, et n'est fait que pour les décisions réellement prises par la table : aucune valeur n'est inventée. »

L'exemple de référence (à connaître par cœur)

Décision du **Carrefour 4** (feu `-29958`) à **07:29:01**, stratégie S3, run du 22/10/2026 :

Élément	Valeur	Ce que ça veut dire
Situation observée	file servie 0 · file ailleurs 6–10 · en mouvement 0 · vert ≤ 10 s	personne n'attend au vert, 6 à 10 véhicules attendent ailleurs
Q(garder) / Q(passer)	−4,651 / −1,481	« passer » a la meilleure valeur
Δ = Q(passer) − Q(garder)	+3,170026	Δ > 0 donc l'agent passe au vert suivant
Essais garder / passer	10 / 79	situation bien connue (≥ 3 essais chacune) : décision de confiance
Valeur de référence	+2,895279	Δ moyen dans les situations comparables
en_mouvement	+0,520247	aucun véhicule ne roule au vert : pousse vers « passer »
file_autre	−0,447999	moins d'attente ailleurs que d'habitude : freine un peu
durée_vert	+0,202498	vert court : pousse un peu vers « passer »
file_servie	0	constante dans le fond : 0 par construction
Vérification	2,895279 + 0,274747 = 3,170026	additivité exacte (erreur 0)

2. Rappel : comment l'agent S3 décide

Q-learning tabulaire. Chacun des 4 feux a sa propre table. La table a été entraînée en simulation pendant **26 épisodes** avec une demande ×5 (graines 101 à 126), puis **gelée** : pendant une démonstration elle n'apprend plus, elle applique ce qu'elle a appris (fichier `qtable_v4_FROZEN_ep26_eval.json`, 1 999 états, dont 341 de confiance).

L'état observé : 5 éléments

Élément	Signification simple	Classes (intervalles)
phase verte	quelle phase du feu est au vert	numéro de phase
`file_servie`	véhicules arrêtés sur les voies qui ont le vert	0 · 1–2 · 3–5 · 6–10 · > 10
`file_autre`	véhicules arrêtés sur une autre phase (qui attendent leur vert)	0 · 1–2 · 3–5 · 6–10 · > 10
`en_mouvement`	véhicules qui roulent sur les voies au vert	0 · 1–2 · 3–5 · > 5
`durée_vert`	depuis combien de temps le vert dure	≤ 10 s · 11–20 s · 21–40 s · > 40 s

Deux actions et une décision

- **garder** le vert, ou **passer** à la phase suivante. La table donne une valeur à chacune : **Q(garder)** et **Q(passer)** (récompense future attendue ; plus elle est haute, mieux c'est).
- On note **Δ = Q(passer) − Q(garder)**. Si Δ > 0, l'agent passe ; sinon il garde. **C'est Δ que SHAP explique.**

Trois garde-fous qui comptent pour l'XAI

Règle	Ce qui se passe	Conséquence pour SHAP
Règle de confiance	La table n'est utilisée que si chaque action a été essayée au moins 3 fois dans cette situation et que les deux valeurs sont différentes.	Décision « table apprise » : SHAP possible
Règle de secours S2	Situation trop peu vue ou jamais vue : on applique la règle S2 (demande / creux).	Pas de SHAP : ce n'est pas la table qui a décidé
Sécurité « vert maximum »	L'agent d'intervention force le changement quand le vert atteint sa durée maximale (et refuse un vert de moins de 5 s).	Pas de SHAP : c'est une règle de sécurité

Le journal des décisions. Chaque changement de phase est enregistré par l'agent d'intervention (`TLSActuator`) avec ce que l'agent a observé : situation, valeurs Q, nombre d'essais, source de la décision. À la fin du run, il est écrit dans `decisions.csv`. Les décisions « garder » ne sont pas enregistrées (rien ne change au feu).

> **IMPORTANT : S0 ET S1 N'ONT PAS DE DÉCISIONS**
> Avec S0 (temps fixe) et S1 (feux « actuated » de SUMO), ce sont les programmes de SUMO qui changent les phases ; aucun agent ne décide, donc la liste des décisions est vide. Ce n'est pas un bug : c'est pour cela que la démonstration XAI se fait avec **S3**.

3. SHAP expliqué simplement

> **L'IMAGE À DONNER**
>
> Quatre joueurs (nos 4 variables) ont gagné ensemble un « gain » : l'écart entre le Δ de cette décision et le Δ habituel. La valeur de **Shapley** (théorie des jeux, Lloyd Shapley, 1953) partage ce gain de façon équitable : on regarde ce que chaque joueur apporte en moyenne quand il rejoint l'équipe, dans tous les ordres possibles. SHAP (*SHapley Additive exPlanations*) applique cette idée aux modèles d'IA.

La formule à comprendre

Δ de la décision = valeur de référence + contribution(file_servie) + contribution(file_autre) + contribution(en_mouvement) + contribution(durée_vert)

- **Valeur de référence (base value)** : le Δ moyen sur un **fond de référence**, c'est-à-dire les situations du même feu et de la même phase verte présentes dans la table, **pondérées par leur nombre de visites** (les situations fréquentes comptent plus).
- **Contribution positive** : la variable pousse vers « passer ». **Négative** : elle pousse vers « garder ».
- **Additivité** : la référence plus la somme des contributions redonne exactement Δ. Nous le vérifions à chaque calcul (tolérance 10^{-9}).

Comment notre moteur calcule (les mots techniques)

- **Exact** : avec 4 variables il n'y a que 2^4 = **16 coalitions** ; on les calcule toutes, sans échantillonnage ni approximation.
- **Interventionnel** : une variable « absente » prend la valeur qu'elle a dans une situation du fond. On obtient un **état hybride** (une partie vient de la décision, une partie du fond) dont on lit le Δ dans la table.
- **Rien n'est inventé** : si un état hybride n'existe pas dans la table, la situation du fond est **exclue** entièrement (et comptée). Les valeurs Q peu fiables (une action essayée moins de 3 fois) sont utilisées telles quelles mais **comptées et signalées**.
- **Vérification** : avant le calcul, le moteur vérifie que les valeurs Q enregistrées dans le journal sont bien celles de la table gelée. Sinon, il refuse.
- **Pourquoi pas la bibliothèque `shap`** : avec 4 variables le calcul exact est immédiat (quelques millisecondes) ; pas de dépendance, pas d'approximation, résultat vérifiable.

Comment lire une contribution

Ce que vous voyez	Ce que vous dites
Barre bleue vers la droite	« Cette variable a poussé la table vers passer au vert suivant, par rapport à d'habitude. »
Barre brune vers la gauche	« Cette variable a freiné : elle poussait vers garder le vert. »
0 (constante)	« Cette variable a la même valeur dans toutes les situations de référence : 0 par construction. Cela ne prouve pas qu'elle n'a pas d'effet. »

> **CE QUE SHAP NE DIT PAS**
>
> SHAP explique la **sortie de la table** (Δ), pas un effet causal sur le trafic, et pas si la décision était bonne. La valeur dépend du fond de référence choisi (voir la section 4).

4. Ce que nous avons réalisé, étape par étape

#	Réalisation	Résultat	Tests
1	Moteur SHAP exact `agents/xai/shap_qtable.py`	Explication exacte d'une décision du journal ; vérifications ; rapport `docs/shap_demo_v2` .	18 tests
2	Analyse de stabilité `agents/xai/shap_stability.py`	Même décision, mêmes 29 situations de référence, deux pondérations : visites (A) et uniforme (B). Rapport `docs/shap_stability_v2` .	11 tests
3	Figures `agents/xai/shap_figures.py`	3 figures (contributions, classement, additivité), FR/EN, PNG + PDF : `docs/shap_visualizations_v2` .	12 tests
4	Revue scientifique	Relecture des figures, explication du changement de signe, limites, plan de validation : `docs/shap_scientific_review_v1` .	calculs reproductibles
5	Étude multi-décision `agents/xai/shap_multi_decision.py`	Les 259 décisions apprises d'un run, regroupées en 32 états, 3 fonds de référence. Résultats préliminaires (voir encadré).	19 tests
6	Page « Explicabilité XAI » dynamique `app/frontend/app/xai/`	Décisions réelles en direct, sélection, SHAP calculé à la demande, raisons de non-admissibilité, états d'attente et d'erreur. Nom de l'application corrigé.	26 tests d'interface
7	Backend XAI `app/backend/xai_api.py`	Routes `/api/xai/*` ; journal complet conservé pendant et après le run ; script de démarrage avec diagnostic.	9 tests API

Résultat de l'analyse de stabilité (exemple de référence)

Variable	A : visites	B : uniforme	Rang A / B
en_mouvement	+0,520247	+0,805710	1 / 1
file_autre	−0,447999	+0,260435	2 / 3 — **signe inversé**
durée_vert	+0,202498	+0,793159	3 / 2
file_servie	0	0	4 / 4 (constante)
Valeur de référence	2,895279	1,310722	Δ identique : 3,170026

POURQUOI FILE_AUTRE CHANGE DE SIGNE (À SAVOIR EXPLIQUER)

Une contribution compare la décision à la **référence**. Avec les visites, la référence est surtout faite de situations où plus de 10 véhicules attendent ailleurs (76 % du poids) : « 6–10 » paraît *peu*, donc la contribution est négative. Avec des poids égaux, les situations avec 0 à 5 véhicules pèsent beaucoup plus : « 6–10 » paraît *beaucoup*, donc la contribution devient positive. **La décision ne change pas** : Δ = +3,170026 dans les deux cas ; seul le point de comparaison change. De plus, le signe positif de B vient surtout de situations peu fiables : la pondération par visites (A) reste la méthode principale.

ÉTUDE MULTI-DÉCISION : RÉSULTATS PRÉLIMINAIRES

Sur 32 états distincts (259 décisions), en comparant visites, uniforme et situations fiables seulement : 12 explications stables, 3 partiellement stables, 17 instables (46,9 %) : lecture fixée à l'avance = « sensible au fond de référence ». Calculé dans le cloud, **pas encore validé sous Windows** : à citer avec cette précaution, ou pas du tout.

5. Comment l'interface affiche l'XAI (architecture)

SUMO simulation du trafic (TraCI), 07:00–08:00 → **Orchestrateur** `run_episode` : 5 agents (collecte, détection, prévision, décision, intervention) → **Agent d'intervention** `TLSActuator` applique la décision et l'écrit dans le journal → **Backend FastAPI** `SimulationManager` garde le journal complet → **Routes XAI** `/api/xai/decisions` `/api/xai/shap` → **Page Next.js** « Explicabilité XAI », actualisée toutes les 2 s

Les trois routes de l'API

Route	Rôle
`GET /api/xai/runs`	Liste des runs enregistrés (dossiers `app_runs` contenant un `decisions.csv`).
`GET /api/xai/decisions`	Décisions du run courant (ou d'un run choisi avec `?run=…`), avec pour chacune : admissible au SHAP ou non, et pourquoi.
`GET /api/xai/shap?index=…`	Calcule à la demande le SHAP exact de la décision n° *index* avec le moteur `shap_qtable.py`. Réponse 422 avec la raison si la décision n'est pas admissible.

D'où viennent les décisions affichées

Moment	Source	Affiché dans la carte « Décisions »
Simulation en cours	Journal complet en mémoire du backend	source : mémoire (simulation en cours)
Simulation terminée	`decisions.csv` du run (complet)	source : decisions.csv
Run enregistré (liste déroulante)	`decisions.csv` de ce run	source : decisions.csv

Les numéros de décision ne changent pas entre « en cours » et « terminée » : on peut sélectionner une décision pendant le run et la retrouver après. Nous avons vérifié que le SHAP d'une même décision est identique avant et après la fin (écart inférieur à 10^{-12}).

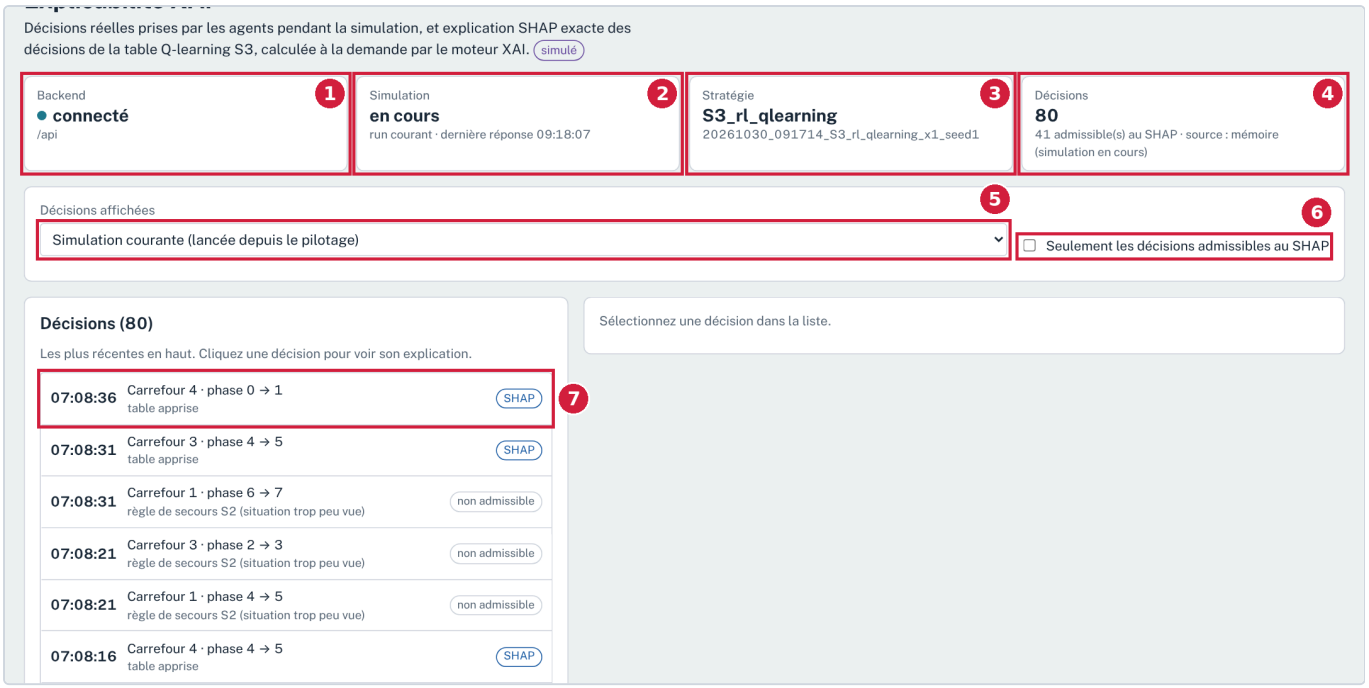
Quelles décisions reçoivent un SHAP

Type de décision	Badge	Raison affichée
S3, source « table apprise », action passer	**SHAP**	calcul exact disponible
S3, règle de secours S2 (trop peu vue / jamais vue)	non admissible	la table n'était pas assez fiable dans cette situation
Règle de sécurité « vert maximum »	non admissible	changement imposé, pas une décision de la table
S2, S2F, max-pressure	non admissible	stratégie sans table Q-learning
S0, S1	(aucune décision)	phases gérées par SUMO

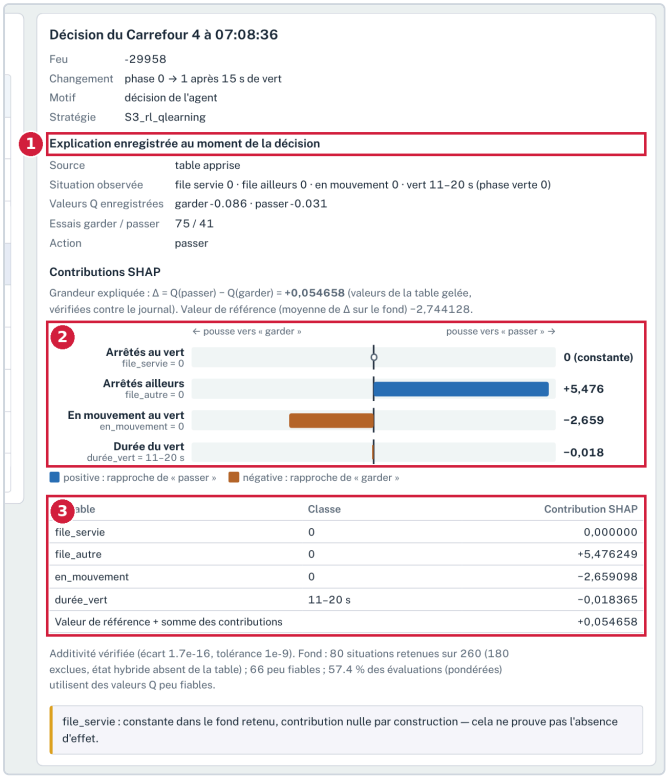
SÉCURITÉ ET HONNÊTETÉ DU CALCUL

- Les noms de runs sont vérifiés : impossible de lire un fichier arbitraire du PC.
- Les valeurs Q du journal doivent correspondre à la table gelée (à 0,0005 près, arrondi du journal) ; sinon : « Vérification refusée », aucune valeur.
- Le graphique est construit uniquement avec les nombres renvoyés par l'API ; aucune image pré-calculée n'est montrée comme si elle concernait la décision courante.

6. La page « Explicabilité XAI » pièce par pièce



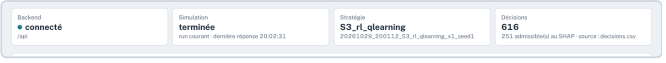
Pendant un run S3 réel. 1 Backend : connecté ou injoignable. 2 Simulation : aucune, en cours, en pause, terminée, arrêtée. 3 Stratégie et nom du run. 4 Nombre de décisions, nombre admissible au SHAP, et source. 5 Choix : simulation courante ou run enregistré. 6 Filtre « seulement admissibles ». 7 Une décision : heure, carrefour, changement de phase, source, badge.



Décision choisie. 1 Explication enregistrée au moment de la décision (situation, valeurs Q, essais). 2 Graphique SHAP calculé par le moteur. 3 Valeurs exactes et vérification « référence + somme = Δ ». En dessous : additivité, taille du fond, part de valeurs peu fiables, avertissement sur les variables constantes.



1 Décision « non admissible ». 2 La raison est écrite en clair, aucune valeur n'est inventée.



Après la fin : « terminée », les décisions restent affichées (source : decisions.csv).



Si le backend ne répond plus : message rouge avec la commande pour le relancer.

7. Préparer la démonstration

La veille (une seule fois, 20 minutes)

Ouvrez PowerShell dans le dossier du projet `C:\Users\ASUS\Intelligent-Traffic-Management-Luxembourg`.

1. **Lancer les tests XAI et API** (sans SUMO) :

   ```
   python -m pytest tests\test_xai_api.py tests\test_xai_shap.py tests\test_xai_shap_stability.py
   tests\test_xai_shap_figures.py tests\test_xai_shap_multi_decision.py -v -ra
   ```

2. **Reconstruire l'interface** (indispensable : sinon l'ancienne page XAI reste affichée). `npm run build` réécrit le dossier `out` : on le sauvegarde d'abord.

   ```
   Copy-Item -Recurse app\frontend\out app\frontend\out_build_20261029
   cd app\frontend
   npm run build
   cd ..\..
   ```

3. **Tester l'interface** (lance un navigateur invisible, ne lance pas SUMO) :

   ```
   python -m pytest tests\test_frontend_smoke.py -v -ra
   ```

4. **Faire une répétition complète** en suivant la section 8, avec la vitesse « Rapide » pour gagner du temps.

Le jour même (10 minutes avant)

1. Fermez les autres programmes lourds. Branchez le chargeur.
2. Dans PowerShell, à la racine du projet, démarrez le backend :

   ```
   powershell -ExecutionPolicy Bypass -File scripts\windows\start_backend_windows.ps1
   ```

   Attendez la ligne `Uvicorn running on http://127.0.0.1:8000`. **Ne fermez pas cette fenêtre** : la fermer arrête l'application. (Commande équivalente : `python -m app.backend.server`, toujours depuis la racine du projet.)
3. Ouvrez Chrome ou Edge sur **http://127.0.0.1:8000/**. Zoom 100 % ou 110 % pour le vidéoprojecteur.
4. Cliquez sur **Explicabilité XAI** : vérifiez « Backend : connecté » et « Simulation : aucune simulation ». Revenez sur **Pilotage**.

> **À ÉVITER PENDANT LA DÉMONSTRATION**
>
> - Ne cochez pas « Ouvrir aussi SUMO-GUI » sauf si vous l'avez testé la veille (la fenêtre SUMO peut s'ouvrir derrière le navigateur ou échouer).
> - Ne cliquez pas deux fois sur « Lancer » : une seule simulation à la fois.
> - Ne relancez pas le backend pendant un run : le run en cours serait arrêté (ses résultats partiels sont quand même écrits).

> **SI LE BACKEND NE DÉMARRE PAS**
>
> Lancez le diagnostic, qui dit exactement ce qui manque (Python, paquets, erreur d'import, port 8000 occupé, interface non construite) :
>
> ```
> powershell -ExecutionPolicy Bypass -File scripts\windows\start_backend_windows.ps1 -CheckOnly
> ```

8. Scénario de démonstration : je clique sur quoi, je dis quoi

Durée : 10 à 12 minutes. Le run « Lente » dure environ 6 minutes : on explique pendant qu'il tourne.

Étape 1 · Présenter le contexte

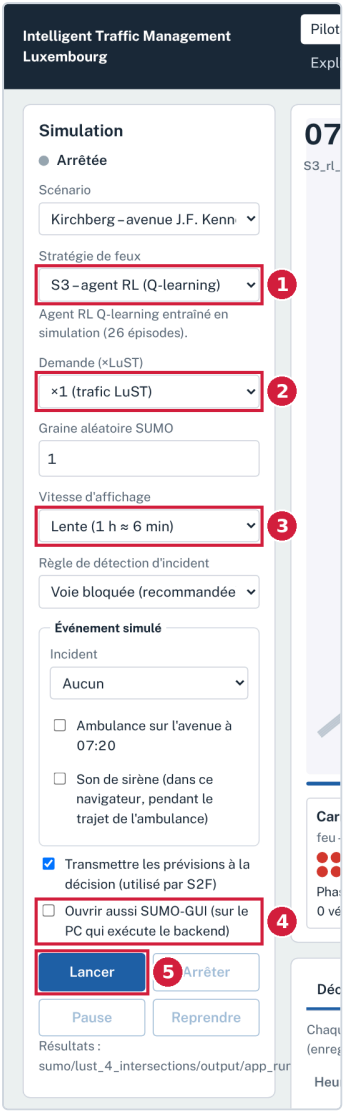
page Pilotage · 1 min

JE CLIQUE

Rien. La page **Pilotage** est ouverte.

JE DIS

« Voici notre application Intelligent Traffic Management Luxembourg. Elle pilote une simulation SUMO de quatre carrefours de l'avenue Kennedy au Kirchberg, entre 7 h et 8 h, avec le trafic du scénario LuST. Toutes les données sont simulées. Je vais lancer l'agent Q-learning S3, puis montrer comment nous expliquons chacune de ses décisions. »



Formulaire de la page Pilotage.

Étape 2 · Configurer et lancer

1 min

JE CLIQUE, DANS L'ORDRE

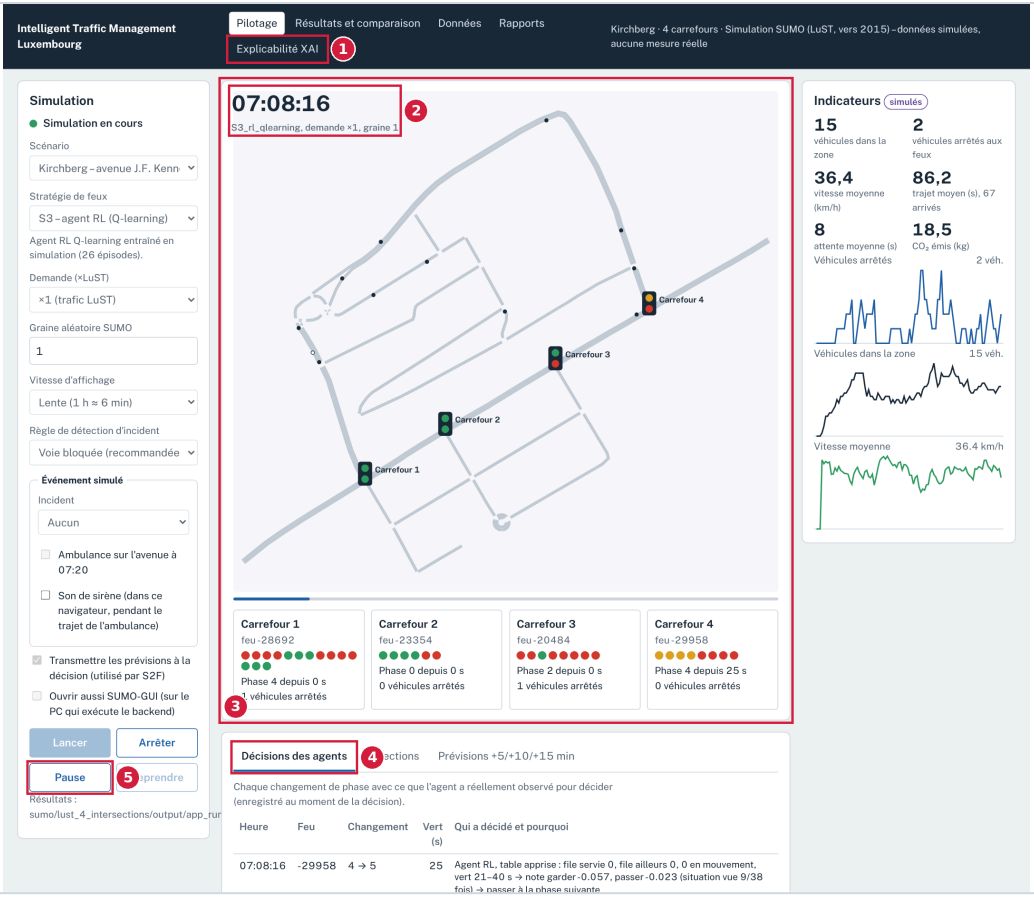
1. **Stratégie de feux** : choisir **S3 – agent RL (Q-learning)**.
2. **Demande** : **×1 (trafic LuST)**. Graine : laisser **1**.
3. **Vitesse d'affichage** : **Lente (1 h ≈ 6 min)** (ou « Rapide » si on manque de temps).

Incident : **Aucun**. Ambulance : **non cochée**.

4. **SUMO-GUI** : laisser **non coché**.
5. Cliquer **Lancer**.

JE DIS

« Je choisis la stratégie S3 : l'agent d'apprentissage par renforcement. Sa table Q a été entraînée pendant 26 épisodes puis gelée : il applique ce qu'il a appris. Demande ×1, c'est le trafic normal du scénario. Je lance. »



Page Pilotage pendant le run. 1 lien Explicabilité XAI · 2 horloge simulée · 3 carte et feux · 4 onglet « Décisions des agents » · 5 Pause.

Étape 3 · Montrer la simulation — 1 min 30

JE MONTRE

2 l'horloge qui avance ; 3 la carte avec les véhicules et les 4 carrefours ; les indicateurs à droite ; 4 l'onglet **Décisions des agents** : lire une ligne « Agent RL, table apprise : … → passer à la phase suivante ».
Option : 5 **Pause** pour figer, puis **Reprendre**.

JE DIS

« Toutes les 5 secondes, l'agent observe chaque feu : véhicules arrêtés au vert, arrêtés ailleurs, en mouvement, durée du vert. Quand il change de phase, il enregistre ce qu'il a vu et ses deux valeurs Q. Mais ces valeurs ne disent pas *pourquoi*. C'est le rôle de la page XAI. »

Étape 4 · Ouvrir la page XAI — 30 s

JE CLIQUE

1 **Explicabilité XAI** dans le menu du haut. La simulation continue : elle tourne dans le backend, pas dans la page.

JE DIS

« La page XAI lit en direct les décisions réelles de la simulation, par l'API du backend. Rien n'est pré-calculé. »

Étape 5 · Lire l'état en haut de la page — 1 min

JE MONTRE (IMAGE SECTION 6)

1 Backend **connecté** ; 2 Simulation **en cours** ; 3 Stratégie **S3_rl_qlearning** ; 4 le nombre de décisions qui augmente, et combien sont admissibles.

JE DIS

« Le compteur augmente pendant la simulation. Seules les décisions prises par la table Q apprise sont admissibles au SHAP : les autres viennent de la règle de secours ou d'une règle de sécurité. »

Étape 6 · Expliquer une décision — 3 min — le cœur de la démo

JE CLIQUE

7 une ligne avec le badge **SHAP** (de préférence avec des barres visibles des deux côtés ; sinon en essayer une autre).
Je montre à droite : 1 l'explication enregistrée, 2 le graphique, 3 le tableau et la ligne « Valeur de référence + somme des contributions ».

JE DIS

« Pour cette décision, l'agent a observé telle situation et a choisi de passer. Le moteur calcule les valeurs de Shapley exactes : 16 coalitions, sans approximation. Chaque barre dit de combien la variable a poussé vers passer (bleu) ou vers garder (brun), par rapport à une situation de référence : le Δ moyen des situations comparables, pondérées par leurs visites. La référence plus la somme des contributions redonne exactement le Δ de la table : c'est la propriété d'additivité, vérifiée à chaque calcul. »

Étape 7 · Montrer une décision non admissible — 1 min

JE CLIQUE

une ligne **non admissible** (« règle de secours S2 » ou « vert maximum »). Puis, si on veut, 6 la case **Seulement les décisions admissibles au SHAP**.

JE DIS

« Ici la table n'était pas assez sûre d'elle (moins de 3 essais d'une action) : c'est la règle de secours S2 qui a décidé. Nous ne calculons donc pas de SHAP : nous refusons d'expliquer avec la table une décision qu'elle n'a pas prise. La raison est affichée. »

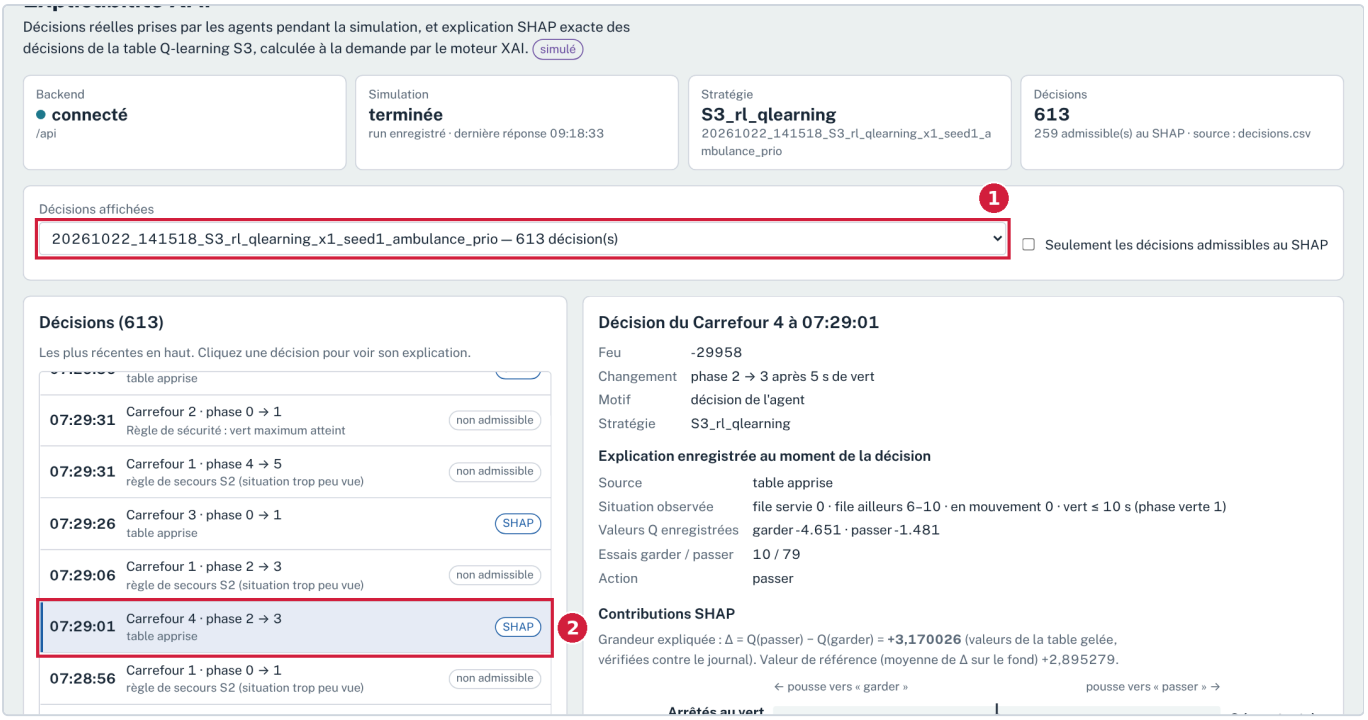
Étape 8 · Fin de la simulation — 30 s

JE MONTRE

La carte « Simulation » passe à **terminée** ; les décisions restent ; la source devient **decisions.csv** ; la décision sélectionnée reste affichée.

JE DIS

« À la fin, les décisions sont relues depuis le fichier decisions.csv du run : elles restent consultables, même après un redémarrage du backend. »



1 liste « Décisions affichées » : choisir le run du 22/10. 2 la décision 07:29:01, Carrefour 4.

Étape 9 · L'exemple de référence et la stabilité

2 min

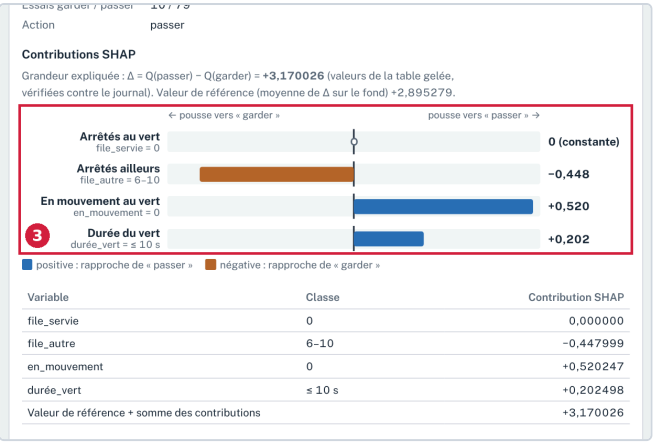
JE CLIQUE

1 la liste **Décisions affichées** → **20261022_141518_S3_rl_qlearning_x1_seed1_ambulance_prio**.

2 descendre dans la liste jusqu'à **07:29:01 · Carrefour 4** (badge SHAP) et cliquer.

JE DIS

« Voici la décision que nous avons étudiée en détail : Δ = +3,17. Le fait qu'aucun véhicule ne roule au vert pousse le plus vers passer (+0,52). Nous avons testé la stabilité : avec une pondération uniforme du fond, la contribution de file_autre change de signe. Ce n'est pas la décision qui change, c'est le point de comparaison. Nous gardons donc la pondération par visites, et nous le signalons comme une limite. »



Le graphique attendu pour 07:29:01 : 0 · −0,448 · +0,520 · +0,202, et la somme +3,170026.

Étape 10 · Conclure — 30 s

OPTION

Choisir le run S1 du 29/10 dans la liste : la page affiche « aucune décision » et explique pourquoi (feux gérés par SUMO).

JE DIS

« En résumé : nous expliquons exactement et honnêtement les décisions de la table Q-learning, en direct dans l'interface. Les limites : une seule méthode de référence, une partie des valeurs Q du fond est peu fiable, et SHAP explique la table, pas un effet causal sur le trafic. »

9. Plan B : si quelque chose ne marche pas

Ce que vous voyez	Cause probable	Ce que vous faites
Message rouge « Backend injoignable »	Fenêtre PowerShell fermée, backend arrêté	Relancer `start_backend_windows.ps1` ; la page se met à jour seule.
Le navigateur n'ouvre pas http://127.0.0.1:8000	Backend pas démarré, ou port 8000 occupé	Lancer le script avec `-CheckOnly` : il nomme le programme qui occupe le port.
La page XAI montre encore les 3 images fixes	Interface pas reconstruite	`npm run build` dans `app\frontend` (section 7), puis recharger la page (Ctrl+F5).
Bouton « Lancer » grisé	Une simulation est déjà active	Cliquer « Arrêter », attendre « arrêtée », puis relancer.
S3 marqué « (indisponible) »	Table Q gelée introuvable	Vérifier `sumo\lust_4_intersections\output\experiments_v1\rl\qtable_v4_FROZEN_ep26_eval.json` .
Erreur au lancement de SUMO	SUMO-GUI coché, ou SUMO mal installé	Décocher SUMO-GUI et relancer. Sinon, faire la démo XAI avec le **run enregistré du 22/10** (étape 9) : tout fonctionne sans lancer SUMO.
Aucune décision dans la page XAI	Stratégie S0 ou S1 lancée	C'est normal (section 2) : relancer avec S3.
Aucune décision avec badge SHAP pour l'instant	Début du run : surtout des règles de secours	Attendre une minute, ou cocher le filtre « Seulement admissibles ».
« Vérification refusée »	Le journal ne correspond pas à la table gelée	Normal et voulu : expliquer que le moteur refuse plutôt que d'inventer.

> **SOLUTION DE SECOURS LA PLUS SÛRE**
>
> Si le temps manque ou si SUMO pose problème : page **Explicabilité XAI** → liste **Décisions affichées** → run du 22/10 → décision 07:29:01 Carrefour 4. Cette partie n'a pas besoin de lancer de simulation.

10. Questions probables du professeur

Question	Réponse courte
Pourquoi SHAP ?	Méthode reconnue et fondée sur la théorie des jeux, avec des propriétés garanties (additivité, équité). Avec 4 variables, on peut la calculer exactement.
Pourquoi seulement S3 ?	C'est le seul agent qui décide avec un modèle appris (la table Q). S2 est une règle déjà lisible ; S0 et S1 ne prennent pas de décision dans l'application.
Comment savez-vous que le calcul est juste ?	Additivité vérifiée à 10^{-9} ; valeurs Q vérifiées contre le journal ; tests sur des tables artificielles dont on connaît le résultat à la main ; comparaison avec un calcul indépendant par permutations.
SHAP prouve-t-il une cause ?	Non. Il décrit comment les variables ont fait bouger la sortie de la table, pas l'effet réel sur le trafic.
Pourquoi file_servie vaut 0 ?	Elle a la même classe (0) dans toutes les situations du fond retenu : 0 par construction, ce qui ne prouve pas qu'elle n'a pas d'effet.
Les explications sont-elles stables ?	Elles dépendent du fond de référence (visites ou uniforme) : nous l'avons mesuré et nous le présentons comme un résultat, pas comme une erreur.
Pourquoi pas les décisions « garder » ?	Elles ne sont pas enregistrées : seuls les changements de phase sont journalisés.
Que se passe-t-il après la simulation ?	Les décisions sont relues depuis decisions.csv ; même numéro, même SHAP qu'en direct.
Les données sont-elles réelles ?	Non : simulation SUMO avec la demande LuST. Aucune mesure réelle dans la zone.

11. Limites à dire honnêtement

- **Fond de référence** : les explications changent selon la pondération (visites ou uniforme) ; nous utilisons les visites comme méthode principale.
- **Valeurs peu fiables** : pour l'exemple, 22 des 29 situations du fond ont une action essayée moins de 3 fois ; leur part est affichée sous le graphique.
- **Entraînement ×5, démonstration ×1** : la table a appris avec une demande ×5 ; les situations typiques peuvent différer.
- **Classes** : les variables sont des intervalles, pas des comptages exacts.
- **Portée** : seules les décisions « passer » de la table S3 sont expliquées ; SHAP n'évalue pas si la décision était bonne.
- **Validation** : l'étude multi-décision et la page dynamique ont été testées dans le cloud ; à valider sous Windows avant de les citer comme définitives.

12. Lexique des mots techniques

Mot	Explication simple
XAI	Explainable AI : méthodes pour expliquer pourquoi une IA prend une décision.
Q-learning tabulaire	Apprentissage par renforcement où l'agent remplit une table : une valeur par situation et par action.
Table Q gelée	Table figée après l'entraînement (26 épisodes) : elle n'apprend plus pendant l'utilisation.
État / situation	Ce que l'agent observe : phase verte + 4 variables en classes.
Q(garder), Q(passer), Δ	Valeurs des deux actions ; Δ = Q(passer) − Q(garder), positif → passer.
Règle de secours S2	Règle simple utilisée quand la table n'a pas assez d'expérience (moins de 3 essais d'une action).
Vert maximum (max_green)	Règle de sécurité qui force le changement de phase au-delà d'une durée.
Valeurs de Shapley / SHAP	Partage équitable de Δ − référence entre les variables, issu de la théorie des jeux.
Coalition	Un sous-ensemble des variables ; 4 variables → 16 coalitions.
Fond de référence	Situations de comparaison : même feu, même phase verte, présentes dans la table, pondérées par les visites.
Valeur de référence	Δ moyen sur le fond de référence (base value).
État hybride	Situation mélangée (variables de la décision + variables du fond) utilisée pendant le calcul.
Interventionnel	Façon de « retirer » une variable : on la remplace par sa valeur dans une situation du fond.
Additivité	Référence + somme des contributions = Δ, exactement.
Admissible	Décision prise par la table apprise, vérifiable : SHAP calculable.
Backend FastAPI	Programme Python qui pilote SUMO et répond aux requêtes de la page (http://127.0.0.1:8000/api).
Route / API	Adresse que la page interroge, par exemple `/api/xai/shap`.
Frontend Next.js	L'interface web ; « export statique » : `npm run build` produit le dossier `out` servi par le backend.
decisions.csv	Fichier du run qui contient toutes les décisions et leur explication enregistrée.
SUMO, TraCI, LuST	Simulateur de trafic ; interface qui le pilote depuis Python ; scénario du trafic de Luxembourg.
Run, graine, demande ×1	Une simulation complète ; nombre qui fixe le hasard ; trafic normal du scénario.